

AI-Based Research Proposal Evaluation System

Comprehensive Technical Defense Document: Tech Stack, Algorithms & AI Models

Project Role: Final Year Engineering Project | **Prepared For:** Final Viva & Evaluation Panel Review | **Date:** September 2026

1. Executive Summary & Core Philosophy

Evaluating research proposals manually is a time-intensive process prone to reviewer fatigue, subjective bias, and delayed funding cycles. The **AI-Based Research Proposal Evaluation System** is designed as an intelligent **Human-in-the-Loop (HITL) Decision Support Platform**. It automates structural format verification, semantic similarity detection, and multi-criteria qualitative evaluation, enabling academic committees and reviewers to make rapid, objective, and well-informed decisions.

***Key Design Principle — Human-in-the-Loop (HITL):** The system does NOT replace human expert reviewers or autonomously accept/reject proposals. Instead, it pre-screens structure, quantifies semantic novelty/overlap, generates multi-criteria baseline scores with rationale, and presents an interactive dashboard for expert reviewers to finalize scores and feedback.*

2. Complete Technical Stack Architecture

The project is architected with a decoupled full-stack modular architecture, combining a high-performance Python FastAPI backend, a modern React + TypeScript frontend, cloud MongoDB database storage, and specialized deep learning / NLP frameworks.

Layer / Subsystem	Technology Stack	Exact Role & Implementation Detail
Backend API Framework	Python 3.11+ FastAPI 0.139.0 Uvicorn 0.51.0	Asynchronous REST API backend routing, data validation using Pydantic v2 schemas, CORS middleware, file uploads, and background tasks.
Frontend Application	React 19.2 TypeScript 5.8 Vite 5.4	Single Page Application (SPA) with strong type-safety, rapid Hot Module Replacement (HMR), component lifecycle management, and custom context state.
Styling & UX	Tailwind CSS 3.4 Lucide React React Hot Toast	Utility-first responsive layout styling, modern visual icon set, real-time notification toasts, and clean dark/light UI palette.
Database & Storage	MongoDB Atlas Motor 3.7 (Async) PyMongo 4.17	Cloud NoSQL database storing user credentials, uploaded proposal metadata, vector embedding lists, review scores, and system notifications.
Security & Auth	JWT (python-jose) Passlib (bcrypt 4.0) OAuth2 Bearer	Stateless authentication using JWT Bearer Tokens (HS256), salted bcrypt password hashing, auth splash rehydration, and Role-Based Access Control (RBAC).

AI / LLM Engine	Google Gemini API google-generativeai 0.8 gemini-3.5-flash	Large Language Model evaluation pipeline extracting structured JSON multi-criteria scorecards, qualitative strengths, weaknesses, and suggestions.
Semantic Vector Engine	Sentence-Transformers PyTorch 2.13 FAISS-CPU 1.14	Pre-trained transformer embeddings generating 384-dimensional dense vectors with FAISS index search for real-time similarity matching.
Document Parsing & NLP	PyMuPDF (fitz 1.28) SpaCy 3.8 Scikit-Learn 1.9	High-speed raw text extraction from PDF documents, text cleaning pipelines (regex whitespace/symbol removal), and Cosine Similarity scoring.
Data Visualization	Recharts 3.10	Interactive analytics charts (Bar charts, Radar score distribution, Status transition pie charts) in Admin and Reviewer dashboards.

3. Algorithms & Mathematical Formulations

The system relies on six core algorithms spanning natural language processing, vector linear algebra, pattern recognition, and multi-criteria decision scoring:

3.1 Cosine Similarity Semantic Overlap Algorithm

To measure conceptual overlap between a newly submitted research proposal and historical proposals stored in MongoDB, both documents are converted into dense vector representations **u** and **v** in 384-dimensional space. The Cosine Similarity algorithm calculates the normalized inner dot product of the two vectors:

Formula: $Similarity(u, v) = (u \cdot v) / (||u|| * ||v||) = (\sum u_i * v_i) / (\sqrt{\sum u_i^2} * \sqrt{\sum v_i^2})$

Output Scaling: The resulting cosine value [-1.0, 1.0] is converted to a percentage: $Score = round(Similarity * 100, 2)$.

3.2 IEEE Standard 8-Section Rule-Based Compliance Algorithm

Validates if uploaded proposal documents conform to standard IEEE academic structure. The algorithm executes high-performance regular expression (RegEx) word boundary checks across 8 mandated sections:

- **IEEE Abstract:** `r'\babstract\b\bexecutive\s+summary\b'`
- **IEEE Index Terms / Keywords:** `r'\bindex\s+terms\b\bkeywords\b'`
- **IEEE Section I (Introduction):** `r'\b(?:section\s+i|1\.)?s*introduction\b'`
- **Problem Statement & Motivation:** `r'\bproblem\s+statement\b\bmotivation\b'`
- **IEEE Section II (Literature Review):** `r'\b(?:section\s+ii|2\.)?s*literature\s+review\b\brelated\s+work[s]?b'`
- **IEEE Section III (Proposed Methodology):** `r'\b(?:section\s+iii|3\.)?s*methodology\b\bssystem\s+model\b'`
- **IEEE Section IV (Expected Outcomes):** `r'\b(?:section\s+iv|4\.)?s*expected\s+outcome[s]?b\bdeliverable[s]?b'`
- **IEEE References & Citations:** `r'\breference[s]?b\b\d+\'`

Compliance Score Formula: $Compliance \% = round((Found\ Sections / 8) * 100)$. Flags `is_valid = True` if all required sections are present.

3.3 Multi-Criteria Evaluation & Weighted Score Aggregation

Google Gemini LLM evaluates proposals along four core qualitative dimensions on an integer scale of 0 to 10: **Novelty (N)**, **Methodology (M)**, **Feasibility (F)**, and **Clarity (C)**. The system aggregates these dimension scores to derive overall baseline metrics:

Overall Mean Score (Out of 10): $S_{overall} = (N + M + F + C) / 4$

Overall Percentage: $Percentage = round(S_{overall} * 10, 2)$

3.4 Text Preprocessing & Cleaning Pipeline Algorithm

Raw PDF text extracted via PyMuPDF often contains line-break hyphens, non-printable font glyphs, and irregular spacing. The NLP cleaning pipeline normalizes text before vector embedding and LLM prompting:

- Whitespace Normalization:** Replaces consecutive control characters/newlines with a single space (`re.sub(r'\s+', ' ', text)`).
- Noise & Glyph Stripping:** Removes invalid unicode glyphs while preserving standard punctuation (`re.sub(r'[^\w\s.,!?\-\:\-]', ' ', text)`).
- Token Truncation Safety:** Truncates text payload to initial 5,000 characters to prevent LLM context overflows while retaining essential proposal content.

3.5 FAISS Indexing & Nearest Neighbor Search Algorithm

Uses Facebook AI Similarity Search (FAISS) IndexFlatL2 and IndexFlatIP data structures to build dense vector index lookup. Allows real-time sub-millisecond similarity comparison over large corpora of historical research proposals.

4. Machine Learning & AI Models Breakdown

The project integrates state-of-the-art Generative AI, Transformer Embeddings, and NLP model pipelines:

Model Name	Model Type / Base Architecture	Key Parameters & Specifications	Specific Project Function & Workflow
Google Gemini (gemini-3.5-flash)	Generative Large Language Model (LLM)	Context Window: 1M+ tokens JSON Mode: Structured schema enforcement	Acts as the multi-criteria evaluator. Processes text and outputs JSON containing Novelty, Methodology, Feasibility, Clarity scores, Strengths, Weaknesses, and Constructive Suggestions.
Sentence-BERT (all-MiniLM-L6-v2)	Sentence Transformer (BERT derivative)	Parameters: 22.7M Output Dimension: 384-d dense vector Max Sequence: 256 tokens	Converts raw proposal abstract and content into 384-dimensional dense semantic vectors to capture deep conceptual context beyond keyword matching.
FAISS Vector Index (faiss-cpu)	Dense Vector Index & Search Library	Metric: L2 Euclidean & Inner Product (IP) Dimension: 384	Stores proposal embedding vectors in an indexed structure for sub-millisecond k-Nearest Neighbor (k-NN) similarity queries.

SpaCy NLP Pipeline (en_core_web_sm)	Convolutional Neural Network NLP Pipeline	Components: Tok2Vec, Parser, NER, Attribute Ruler	Performs tokenization, sentence segmentation, lemmatization, and text cleaning on raw extracted PDF strings.
XGBoost Classifier	Gradient Boosted Decision Trees (GBDT)	Objective: Multi-class classification & score regression	Capability module setup for supervised scoring refinement and secondary proposal rank classification.

5. End-to-End Execution Flow

When a Researcher submits a proposal PDF, the system executes the following synchronous & asynchronous processing pipeline:

- PDF Upload & Ingestion:** Frontend sends PDF multipart file to /proposal/upload. PyMuPDF extracts raw text.
- Text Cleaning & NLP Normalization:** SpaCy & Regex strip formatting noise, blank lines, and non-ASCII glyphs.
- IEEE Structural Audit:** 8-section RegEx matcher audits document formatting and generates section compliance report.
- Dense Embedding Generation:** Sentence-BERT (all-MiniLM-L6-v2) converts cleaned text into a 384-d vector.
- Vector Similarity Check:** Cosine Similarity & FAISS index search query past proposals in MongoDB Atlas to calculate semantic overlap %.
- Gemini LLM Multi-Criteria Scoring:** Gemini API evaluates text payload and returns structured JSON scorecard with qualitative feedback.
- MongoDB Atlas Persistence:** Complete proposal object (metadata, IEEE check, similarity score, Gemini evaluation) is saved.
- Role-Based Dashboards & Real-time Toast/Notifications:** Admin/Reviewers receive unread header notifications and review proposals.

6. Viva Defense Cheatsheet for Tomorrow's Review

Quick reference answers for potential technical questions from the project evaluation panel:

Q1: Why use Sentence-BERT instead of traditional TF-IDF or Word2Vec?	Answer: TF-IDF only matches literal keywords and fails when proposals use synonyms (e.g., 'machine learning' vs 'deep neural networks'). Word2Vec averages word vectors, losing sentence context. Sentence-BERT uses transformer self-attention to capture full contextual semantic meaning in a 384-d dense embedding space.
Q2: Why Google Gemini instead of OpenAI GPT-4?	Answer: Gemini 3.5 Flash offers extremely low latency, superior native JSON schema output enforcement (preventing malformed parsing errors), cost-efficiency, and a 1M+ token context window suitable for lengthy academic proposals.
Q3: How does the system handle AI hallucination or biased scoring?	Answer: Through our Human-in-the-Loop (HITL) design. AI scores serve solely as an automated baseline. Human expert reviewers inspect the proposal, review AI flags, and submit final authoritative human scores and recommendations.
Q4: How is IEEE section validation implemented?	Answer: It uses high-performance regex pattern matching across 8 mandatory IEEE sections (Abstract, Index Terms, Section I-IV, Citations [1]). It runs in milliseconds without requiring heavy ML inference.

Q5: How is database scalability ensured for vector searches?	Answer: Proposal embeddings (384-float arrays) are stored in MongoDB Atlas and indexed via FAISS (`faiss-cpu`), allowing logarithmic ($O(\log N)$) search time as submission count grows.
Q6: Why TypeScript instead of JavaScript for the Frontend?	Answer: JavaScript can run everything TS does (TS compiles to JS). However, TypeScript was selected for high engineering quality: (1) Compile-time Type Safety: Catches runtime bugs (e.g. undefined is not an object) before deployment. (2) Backend Contract Alignment: TS Interfaces directly match FastAPI Pydantic schemas (Proposal, Evaluation, User). (3) Maintainability & Refactoring: Allows safe refactoring across complex multi-role dashboards (Researcher, Reviewer, Admin).